

AskObi API – Architecture & Authentication

1. Overview

The AskObi API is a **FastAPI** application providing secure authentication and session management. It uses **JWT access tokens** and **rotating refresh tokens**, with per-device sessions tracked in **Postgres** and cached in **Redis** for fast validation.

2. System Architecture

- **Framework:** FastAPI (`main.py`)
 - **Authentication:**
 - Access tokens (short-lived JWTs)
 - Rotating refresh tokens (long-lived, hashed in DB)
 - Per-device sessions stored in Postgres
 - Redis cache for quick session validation
 - **Database:** Postgres via SQLAlchemy/Advanced Alchemy (async sessions)
 - **Cache:** Redis
 - **Configuration:**
 - Defaults: `127.0.0.1:6383`, db `0`
 - Docker Compose:
 - Redis → `6383:6379`
 - Postgres → `5458:5432`
-

3. Data Model

users

Column	Type	Purpose
<code>email</code>	String	Unique identifier
<code>hashed_password</code>	String	Secure password storage
<code>is_active</code>	Boolean	Account status
<code>is_superuser</code>	Boolean	Admin flag
<code>token_version</code>	Integer	Used to invalidate all tokens (logout-all, password change)
<code>preferences</code>	JSON	User preferences

user_sessions

Column	Type	Purpose
<code>user_id</code>	UUID	Linked user

Column	Type	Purpose
session_id	UUID	Unique session
refresh_token_hash	String	Hashed refresh token
device_name	String	Device metadata
ip_address	String	Client IP
user_agent	String	Browser/agent info
is_active	Boolean	Session status
expires_at	DateTime	Expiration timestamp

4. Core Components

- **Routers (router.py)**
 - /register, /login, /refresh, /logout, /logout-all, /sessions, /me
- **Dependencies (dependencies.py)**
 - get_db() → async DB session
 - get_current_user() → decode JWT, check Redis, validate token_version
 - require_active_user() → ensure active status
- **Services**
 - AuthService → session lifecycle, token rotation, logout-all
 - UserService → registration, authentication
- **Repositories**
 - user_repository.py → CRUD for users/sessions, token_version bump
- **Tokens (tokens.py)**
 - Access JWT: sub, sid, auth_time, exp, type=access
 - Refresh token: random string, hashed in DB
- **Redis Helper (redis.py)**
 - Keys: session:{user_id}:{session_id}
 - Value: JSON {user_id, token_version, device/ip/UA, created_at}
 - TTL: refresh lifetime

5. Authentication Flows

- **Login**
 1. Validate credentials

2. Create `user_sessions` row
3. Set Redis key with TTL
4. Return access + refresh tokens

- **Protected Request**

1. Decode access JWT
2. Check Redis session
3. Validate user + `token_version`
4. Grant access

- **Refresh**

1. Hash provided refresh token
2. Lookup `user_sessions`
3. Verify active + not expired
4. Redis + `token_version` check
5. Rotate refresh token
6. Renew Redis TTL
7. Issue new tokens

- **Logout (current session)**

- Delete Redis key
- Delete session row

- **Logout-all**

- Delete all Redis keys for user
- Delete all session rows
- Increment `token_version`

- **Session Management**

- List sessions from DB
- Revoke → delete Redis key + session row

6. Configuration Notes

- **Redis URL**

- Host API → `redis://127.0.0.1:6383/0`
- Inside Compose → `redis://redis:6379/0`

- **.env Defaults**

- `REDIS_HOST=localhost`
 - `REDIS_PORT=6383`
 - On Windows, prefer `127.0.0.1`
-

7. Operational Considerations

- Ensure Redis pool is initialized
- Redis errors in `get_session` are swallowed → add logging
- **Token lifetimes:**
 - Access: 30 minutes
 - Refresh: 30 days
- **Security:**
 - Refresh tokens stored only hashed
 - `token_version` bump protects against stolen tokens